

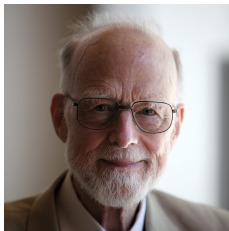
COMP2111 Week 8

Term 1, 2023

Hoare Logic

Sir Tony Hoare

- Pioneer in formal verification
- Invented: Quicksort,
- the null reference (called it his “billion dollar mistake”)
- CSP (formal specification language), and
- Hoare Logic



Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Imperative Programming

imperō

Definition

Imperative programming is where programs are described as a series of *statements* or commands to manipulate mutable *state* or cause externally observable *effects*.

States may take the form of a *mapping* from variable names to their values, or even a model of a CPU state with a memory model (for example, in an *assembly language*).

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.

$\mathcal{L}$: A simple imperative programming language

Consider the vocabulary of basic arithmetic:

- Constant symbols: $0, 1, 2, \dots$
- Function symbols: $+, *, \dots$
- Predicate symbols: $<, \leq, \geq, |, \dots$
- An **(arithmetic) expression** is a term over this vocabulary.
- A **boolean expression** is a predicate formula over this vocabulary.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

The language $\mathcal{L}$

The language $\mathcal{L}$ is a simple imperative programming language made up of four statements:

Assignment: $x := e$

where x is a variable and e is an arithmetic expression.

Sequencing: $P; Q$

Conditional: if g then P else Q fi

where g is a boolean expression.

While: while g do P od

Factorial in $\mathcal{L}$

Example

```
 $i := 0;$   
 $m := 1;$   
while  $i < N$  do  
   $i := i + 1;$   
   $m := m * i$   
od
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Hoare Logic

We are going to define what's called a *Hoare Logic* for $\mathcal{L}$ to allow us to prove properties of our program.

We write a *Hoare triple* judgement as:

$$\{\varphi\} P \{\psi\}$$

Where φ and ψ are logical formulae about states, called *assertions*, and P is a program. This triple states that if the program P terminates and it successfully evaluates from a starting state satisfying the *precondition* φ , then the result state will satisfy the *postcondition* ψ .

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$
$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

Hoare triple: Examples

Example

$$\{(x = 0)\} x := 1 \{(x = 1)\}$$

$$\{(x = 499)\} x := x + 1 \{(x = 500)\}$$

$$\{(x > 0)\} y := 0 - x \{(y < 0) \wedge (x \neq y)\}$$

Hoare triple: Factorial Examples

Example

```
{N ≥ 0}  
i := 0;  
m := 1;  
while i < N do  
  i := i + 1;  
  m := m * i  
od  
{m = N!}
```

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Motivation

Question

We know what we want informally; how do we establish when a triple is valid?

- Develop a semantics, OR
- Derive the triple in a syntactic manner (i.e. Hoare proof)

Hoare logic consists of one axiom and four inference rules for deriving Hoare triples.

Assignment

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{assign})$$

Intuition:

If x has property φ *after* executing the assignment; then e must have property φ *before* executing the assignment

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{ \quad \quad \quad \} x := y \{(x = y)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$
$$\{(y = y)\} x := y \{(x = y)\}$$

Assignment: Example

Example

$\{(y = 0)\} x := y \{(x = 0)\}$

$\{(y = y)\} x := y \{(x = y)\}$

$\{ \quad \quad \} x := 1 \{(x < 2)\}$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\}$$

Assignment: Example

Example

$$\{(y = 0)\} x := y \{(x = 0)\}$$

$$\{(y = y)\} x := y \{(x = y)\}$$

$$\{(1 < 2)\} x := 1 \{(x < 2)\}$$

$$\{(y = 3)\} x := y \{(x > 2)\} \quad \textit{Problem!}$$

Sequence

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Intuition:

If the postcondition of P matches the precondition of Q we can sequentially combine the two program fragments

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ \quad \} \quad \{ \quad \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{ \quad \} x := 0 \{ (x = 0) \} \quad \{ (x = 0) \} y := 0 \{ (x = y) \}}{\{ \quad \} x := 0; y := 0 \{ (x = y) \}} \quad (\text{seq})$$

Sequence: Example

Example

$$\frac{\{(0 = 0)\} x := 0 \{(x = 0)\} \quad \{(x = 0)\} y := 0 \{(x = y)\}}{\{(0 = 0)\} x := 0; y := 0 \{(x = y)\}} \quad (\text{seq})$$

Conditional

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Intuition:

- When a conditional is executed, either P or Q will be executed.
- If ψ is a postcondition of the conditional, then it must be a postcondition of *both* branches
- Likewise, if φ is a precondition of the conditional, then it must be a precondition of both branches
- Which branch gets executed depends on g , so we can assume g to be a precondition of P and $\neg g$ to be a precondition of Q .

While

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Intuition:

- φ is a **loop invariant**. It must be both a pre- and postcondition of P , so that sequences of P s can be run together.
- If the while loop terminates, g cannot hold.

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Intuition:

- Adding assertions to the precondition makes it more likely the postcondition will be reached
- Removing assertions from the postcondition makes it more likely the postcondition will be reached
- If you can reach the postcondition initially, then you can reach it in the more likely scenario

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Back to Assignment Example

Example

$\{(y = 3)\} x := y \{(x > 2)\}$ *Problem!*

$\{(y = 3)\} x := y \{(x > 2)\} (assign, cons)$
 $\{(y > 2)\} x := y \{(x > 2)\} (assign)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$
 $\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$$\begin{array}{l}
 \{N \geq 0\} \\
 \quad i := 0; \\
 \quad m := 1; \\
 \{m = i! \wedge N \geq 0\} \\
 \text{while } i < N \text{ do} \\
 \quad i := i + 1; \\
 \quad m := m \times i \\
 \text{od } \{m = i! \wedge N \geq 0 \wedge i = N\} \\
 \{m = N!\}
 \end{array}
 \qquad
 \begin{array}{l}
 \frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \\
 \\
 \frac{}{\{\varphi[x := e]\} x := e \{\varphi\}} \\
 \\
 \frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \\
 \\
 \frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}} \\
 \\
 \frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}
 \end{array}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$
 $m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1;$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{N \geq 0\}$

$i := 0;$

$m := 1; \{m = i! \wedge N \geq 0\}$

$\{m = i! \wedge N \geq 0\}$

while $i < N$ do $\{m = i! \wedge N \geq 0 \wedge i < N\}$

$\{m \times (i + 1) = (i + 1)! \wedge N \geq 0\}$

$i := i + 1;$

$\{m \times i = i! \wedge N \geq 0\}$

$m := m \times i$

$\{m = i! \wedge N \geq 0\}$

od $\{m = i! \wedge N \geq 0 \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
    i := 0;
    {1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
    {m = i! ∧ N ≥ 0}
    while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
        {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
        i := i + 1;
        {m × i = i! ∧ N ≥ 0}
        m := m × i
        {m = i! ∧ N ≥ 0}
    od {m = i! ∧ N ≥ 0 ∧ i = N}
    {m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
    i := 0; {1 = i! ∧ N ≥ 0}
{1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
{m = i! ∧ N ≥ 0}
while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
    {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
    i := i + 1;
    {m × i = i! ∧ N ≥ 0}
    m := m × i
    {m = i! ∧ N ≥ 0}
od {m = i! ∧ N ≥ 0 ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{N ≥ 0}
{1 = 0! ∧ N ≥ 0} i := 0; {1 = i! ∧ N ≥ 0}
{1 = i! ∧ N ≥ 0} m := 1; {m = i! ∧ N ≥ 0}
{m = i! ∧ N ≥ 0}
while i < N do {m = i! ∧ N ≥ 0 ∧ i < N}
  {m × (i + 1) = (i + 1)! ∧ N ≥ 0}
  i := i + 1;
  {m × i = i! ∧ N ≥ 0}
  m := m × i
  {m = i! ∧ N ≥ 0}
od {m = i! ∧ N ≥ 0 ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{ if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}}$$

$$\frac{\{\varphi\} P \{\alpha\} \quad \{\alpha\} Q \{\psi\}}{\{\varphi\} P; Q \{\psi\}}$$

$$\frac{\varphi' \Rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \Rightarrow \psi'}{\{\varphi'\} P \{\psi'\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

Practice Exercise

Example

```
 $m := 1;$   
 $n := 1;$   
 $i := 1;$   
while  $i < N$  do  
   $t := m;$   
   $m := n;$   
   $n := m + t;$   
   $i := i + 1$   
od
```

- What does this $\mathcal{L}$ program P compute?
- What is a valid Hoare triple $\{\varphi\}P\{\psi\}$ of this program?
- Prove using the inference rules and consequence axiom that this Hoare triple is valid.

Summary

- $\mathcal{L}$: A simple imperative programming language
- Hoare triples (SYNTAX)
- Hoare logic (PROOF)
- Semantics for Hoare logic

Recall

If R and S are binary relations, then the **relational composition** of R and S , $R; S$ is the relation:

$$R; S := \{(a, c) : \exists b \text{ such that } (a, b) \in R \text{ and } (b, c) \in S\}$$

If $R \subseteq A \times B$ is a relation, and $X \subseteq A$, then the **image of X under R** , $R(X)$ is the subset of B defined as:

$$R(X) := \{b \in B : \exists a \text{ in } X \text{ such that } (a, b) \in R\}.$$

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

How do we determine when $\{\varphi\} P \{\psi\}$ is **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

Informal semantics

Hoare logic gives a proof of $\{\varphi\} P \{\psi\}$, that is: $\vdash \{\varphi\} P \{\psi\}$
(axiomatic semantics)

How do we determine when $\{\varphi\} P \{\psi\}$ is **valid**, that is:
 $\models \{\varphi\} P \{\psi\}$?

If φ holds in a state of some computational model
then ψ holds in the state reached after a successful execution of P .

Informal semantics: Programs

What is a program?

Informal semantics: Programs

What is a program?

A function mapping system states to system states

Informal semantics: Programs

What is a program?

A partial function mapping system states to system states

Informal semantics: Programs

What is a program?

A relation between system states

Informal semantics: States

What is a state of a computational model?

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols

Informal semantics: States

What is a state of a computational model?

Two approaches:

- Concrete: from a physical perspective
 - States are memory configurations, register contents, etc.
 - Store of variables and the values associated with them
- Abstract: from a mathematical perspective
 - The pre-/postcondition predicates *hold* in a state
 - ⇒ States are **logical interpretations** (Model + Environment)
 - There is only one model of interest: standard interpretations of arithmetical symbols
 - ⇒ States are fully determined by **environments**
 - ⇒ States are functions that map variables to values

Informal semantics: **States**

State space (ENV)

$x \leftarrow 0$
 $y \leftarrow 0$
 $z \leftarrow 0$

$x \leftarrow 3$
 $y \leftarrow 2$
 $z \leftarrow 1$

$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 1$

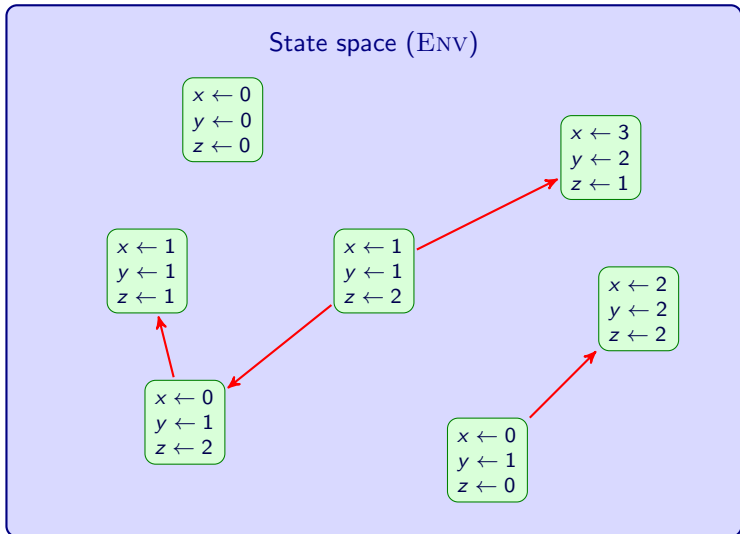
$x \leftarrow 1$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 2$
 $y \leftarrow 2$
 $z \leftarrow 2$

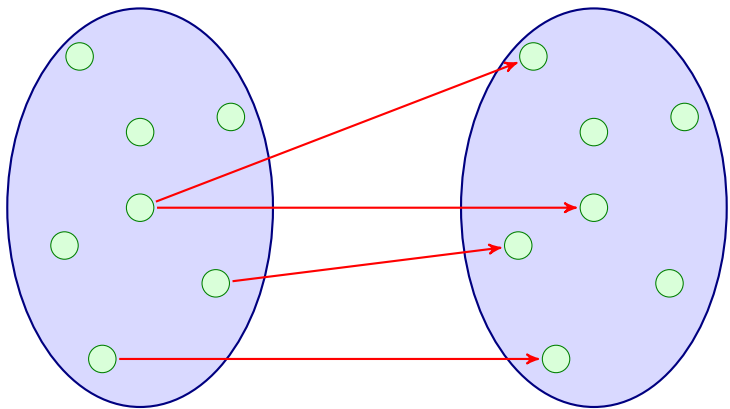
$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 2$

$x \leftarrow 0$
 $y \leftarrow 1$
 $z \leftarrow 0$

Informal semantics: **States** and **Programs**



Informal semantics: **States** and **Programs**



Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to numeric values. We denote by ENV the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

Semantics for $\mathcal{L}$

An **environment** or **state** is a function from variables to numeric values. We denote by ENV the set of all environments.

NB

An environment, η , assigns a numeric value $\llbracket e \rrbracket^\eta$ to all expressions e , and a boolean value $\llbracket b \rrbracket^\eta$ to all boolean expressions b .

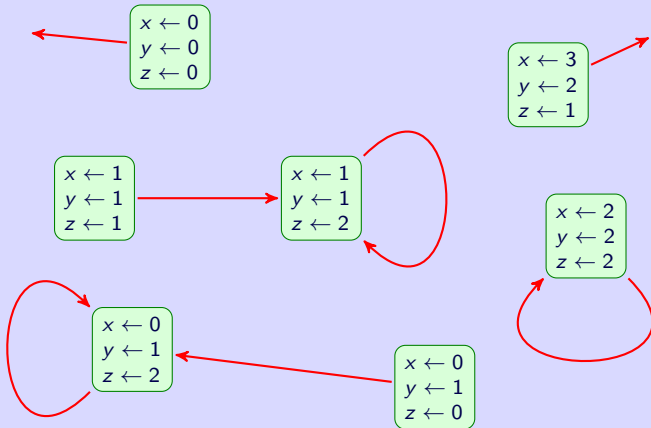
Given a program P of $\mathcal{L}$, we define $\llbracket P \rrbracket$ to be a **binary relation** on ENV in the following manner...

Assignment

$(\eta, \eta') \in \llbracket x := e \rrbracket$ if, and only if $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Assignment: $\llbracket z := 2 \rrbracket$

State space (ENV)



Sequencing

$$\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$$

where, on the RHS, ; is relational composition.

Conditional, first attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \begin{cases} \llbracket P \rrbracket & \text{if } \llbracket b \rrbracket^\eta = \text{true} \\ \llbracket Q \rrbracket & \text{otherwise.} \end{cases}$$

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Detour: Predicates as programs

A boolean expression b defines a subset (or unary relation) of ENV :

$$\langle b \rangle = \{\eta : \llbracket b \rrbracket^\eta = \text{true}\}$$

This can be extended to a binary relation (i.e. a program):

$$\llbracket b \rrbracket = \{(\eta, \eta) : \eta \in \langle b \rangle\}$$

Intuitively, b corresponds to the program

if b then skip else $\perp$ fi

Conditional, better attempt

$$\llbracket \text{if } b \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket b; P \rrbracket \cup \llbracket \neg b; Q \rrbracket$$

While

while b do P od

- Do 0 or more executions of P while b holds
- Terminate when b does not hold

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

While

while b do P od

- Do 0 or more executions of $(b; P)$
- Terminate with an execution of $\neg b$

How to do “0 or more” executions of $(b; P)$?

Transitive closure

Given a binary relation $R \subseteq E \times E$, the *transitive closure* of R , R^* is defined to be the limit of the sequence

$$R^0 \cup R^1 \cup R^2 \dots$$

where

- $R^0 = \Delta$, the diagonal relation
- $R^{n+1} = R^n; R$

NB

- R^* is the smallest transitive relation which contains R
- Related to the Kleene star operation seen in languages: Σ^*

Transitive closure

Given a binary relation $R \subseteq E \times E$, the *transitive closure* of R , R^* is defined to be the limit of the sequence

$$R^0 \cup R^1 \cup R^2 \dots$$

where

- $R^0 = \Delta$, the diagonal relation
- $R^{n+1} = R^n; R$

NB

- R^* is the *smallest transitive relation* which contains R
- Related to the Kleene star operation seen in languages: Σ^*

Technically, R^* is the **least-fixed point** of $f(X) = X \cup X; R$

While

$$\llbracket \text{while } b \text{ do } P \text{ od} \rrbracket = \llbracket b; P \rrbracket^*; \llbracket \neg b \rrbracket$$

- Do 0 or more executions of $(b; P)$
- Conclude with an execution of $\neg b$

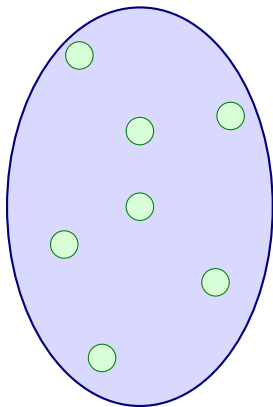
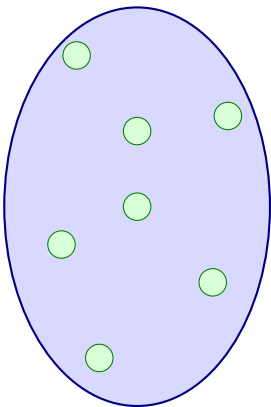
Validity

A Hoare triple is **valid**, written $\models \{\varphi\} P \{\psi\}$ if

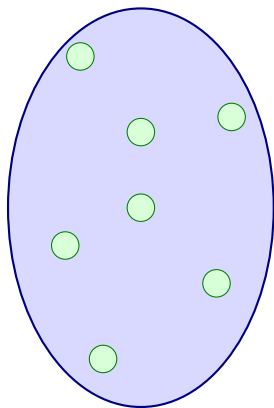
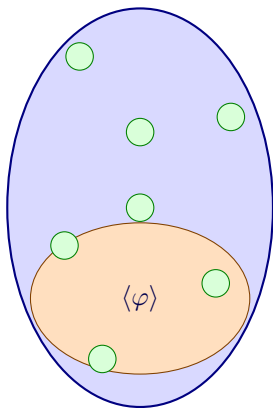
$$\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle.$$

That is, the relational image under $\llbracket P \rrbracket$ of the set of states where φ holds is contained in the set of states where ψ holds.

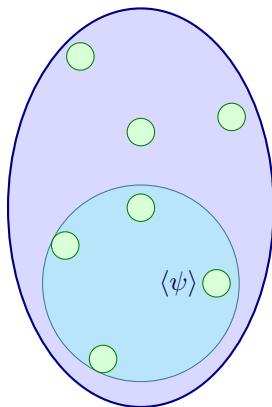
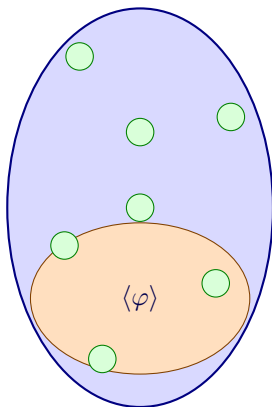
Validity



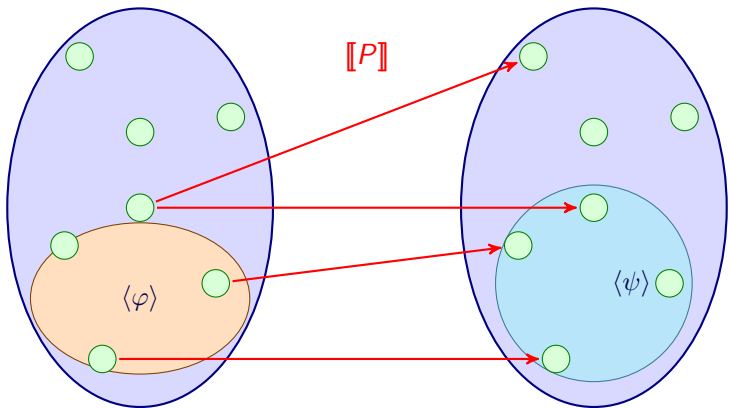
Validity



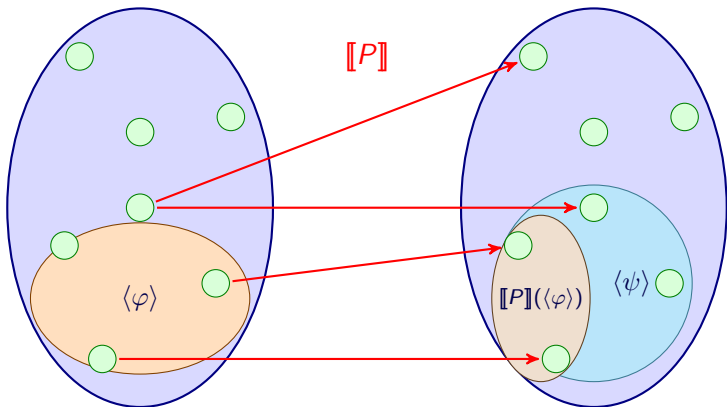
Validity



Validity



Validity



Soundness of Hoare Logic

Hoare Logic is **sound** with respect to the semantics given. That is,

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (a):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (a):

$$\begin{aligned} y \in R(A) &\Leftrightarrow \exists x \in A \text{ such that } (x, y) \in R \\ &\Rightarrow \exists x \in B \text{ such that } (x, y) \in R \\ &\Leftrightarrow y \in R(B) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (b):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (b):

$$\begin{aligned} y \in R(A) \cup S(A) &\Leftrightarrow y \in R(A) \text{ or } y \in S(A) \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } \exists x \in A \text{ s.t. } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in R \text{ or } (x, y) \in S \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, y) \in (R \cup S) \\ &\Leftrightarrow y \in (R \cup S)(A) \end{aligned}$$

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ *If $A \subseteq B$ then $R(A) \subseteq R(B)$*
- Ⓑ *$R(A) \cup S(A) = (R \cup S)(A)$*
- Ⓒ *$R(S(A)) = (S; R)(A)$*

Proof (c):

Some results on relational images

Lemma

For any binary relations $R, S \subseteq X \times Y$ and subsets $A, B \subseteq X$:

- Ⓐ If $A \subseteq B$ then $R(A) \subseteq R(B)$
- Ⓑ $R(A) \cup S(A) = (R \cup S)(A)$
- Ⓒ $R(S(A)) = (S; R)(A)$

Proof (c):

$$\begin{aligned} z \in R(S(A)) &\Leftrightarrow \exists y \in S(A) \text{ s.t. } (y, z) \in R \\ &\Leftrightarrow \exists x \in A, y \in S(A) \text{ s.t. } (x, y) \in S \text{ and } (y, z) \in R \\ &\Leftrightarrow \exists x \in A \text{ s.t. } (x, z) \in (S; R) \\ &\Leftrightarrow z \in (S; R)(A) \end{aligned}$$

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Proof:

Some results on relational images

Corollary

If $R(A) \subseteq A$ then $R^(A) \subseteq A$*

Proof:

$$R(A) \subseteq A \Rightarrow R^{i+1}(A) = R^i(R(A)) \subseteq R^i(A)$$

$$\Rightarrow R^{i+1}(A) \subseteq R(A) \subseteq A$$

$$\text{So } R^*(A) = \left(\bigcup_{i=0}^{\infty} R^i \right) (A)$$

$$= \bigcup_{i=0}^{\infty} R^i(A)$$

$$\subseteq A$$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Proof:

By induction on the structure of the proof.

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \ x := e \ \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \ x := e \ \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

Base case: Assignment rule

$$\frac{}{\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}} \quad (\text{ass})$$

Need to show $\{\varphi[e/x]\} \textcolor{blue}{x} := \textcolor{blue}{e} \{\varphi\}$ is always valid. That is,

$$\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle.$$

Observation: $\llbracket \varphi[e/x] \rrbracket^\eta = \llbracket \varphi \rrbracket^{\eta'}$ where $\eta' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$

So if $\eta \in \langle \varphi[e/x] \rangle$ then $\eta' \in \langle \varphi \rangle$

Recall: $(\eta, \eta'') \in \llbracket x := e \rrbracket$ if and only if $\eta'' = \eta[x \mapsto \llbracket e \rrbracket^\eta]$,

So $\llbracket x := e \rrbracket(\eta) \in \langle \varphi \rangle$ for all $\eta \in \langle \varphi[e/x] \rangle$

So $\llbracket x := e \rrbracket(\langle \varphi[e/x] \rangle) \subseteq \langle \varphi \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see **Lemma 1(c)**)

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see **Lemma 1(c)**)

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

Inductive case 1: Sequence rule

$$\frac{\{\varphi\} P \{\psi\} \quad \{\psi\} Q \{\rho\}}{\{\varphi\} P; Q \{\rho\}} \quad (\text{seq})$$

Assume $\{\varphi\} P \{\psi\}$ and $\{\psi\} Q \{\rho\}$ are valid. Need to show that $\{\varphi\} P; Q \{\rho\}$ is valid.

Recall: $\llbracket P; Q \rrbracket = \llbracket P \rrbracket; \llbracket Q \rrbracket$

So: $\llbracket P; Q \rrbracket(\langle \varphi \rangle) = \llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle))$ (see Lemma 1(c))

By IH: $\llbracket P \rrbracket(\langle \varphi \rangle) \subseteq \langle \psi \rangle$ and $\llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$

So: $\llbracket Q \rrbracket(\llbracket P \rrbracket(\langle \varphi \rangle)) \subseteq \llbracket Q \rrbracket(\langle \psi \rangle) \subseteq \langle \rho \rangle$ (see Lemma 1(a))

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- a) $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- b) $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (a):

$$\begin{aligned}\eta' \in \llbracket \varphi \rrbracket(X) &\Leftrightarrow \exists \eta \in X \text{ s.t. } (\eta, \eta') \in \llbracket \varphi \rrbracket \\ &\Leftrightarrow \exists \eta \in X \text{ s.t. } \eta = \eta' \text{ and } \eta \in \langle \varphi \rangle \\ &\Leftrightarrow \eta' \in X \cap \langle \varphi \rangle\end{aligned}$$

Two more useful results

Lemma

For $R \subseteq \text{ENV} \times \text{ENV}$, predicates φ and ψ , and $X \subseteq \text{ENV}$:

- Ⓐ $\llbracket \varphi \rrbracket(X) = \langle \varphi \rangle \cap X$
- Ⓑ $R(\langle \varphi \wedge \psi \rangle) = (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle)$

Proof (b):

$$\langle \varphi \wedge \psi \rangle = \langle \varphi \rangle \cap \langle \psi \rangle = \llbracket \varphi \rrbracket(\langle \psi \rangle)$$

$$\begin{aligned} \text{So } R(\langle \varphi \wedge \psi \rangle) &= R(\llbracket \varphi \rrbracket(\langle \psi \rangle)) \\ &= (\llbracket \varphi \rrbracket; R)(\langle \psi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle)$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \end{aligned}$$

Inductive case 2: Conditional rule

$$\frac{\{\varphi \wedge g\} P \{\psi\} \quad \{\varphi \wedge \neg g\} Q \{\psi\}}{\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \quad (\text{if})$$

Assume $\{\varphi \wedge g\} P \{\psi\}$ and $\{\varphi \wedge \neg g\} Q \{\psi\}$ are valid. Need to show that $\{\varphi\} \text{if } g \text{ then } P \text{ else } Q \text{ fi } \{\psi\}$ is valid.

Recall: $\llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket = \llbracket g; P \rrbracket \cup \llbracket \neg g; Q \rrbracket$

$$\begin{aligned} & \llbracket \text{if } g \text{ then } P \text{ else } Q \text{ fi} \rrbracket(\langle \varphi \rangle) \\ &= \llbracket g; P \rrbracket(\langle \varphi \rangle) \cup \llbracket \neg g; Q \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(b)}) \\ &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \cup \llbracket Q \rrbracket(\langle \neg g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)}) \\ &\subseteq \langle \psi \rangle \quad (\text{by IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that
 $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\llbracket g; P \rrbracket(\langle \varphi \rangle) = \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) \quad (\text{see Lemma 2(b)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) = \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) \quad (\text{see Lemma 1(c)})$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \end{aligned}$$

Inductive case 3: While rule

$$\frac{\{\varphi \wedge g\} P \{\varphi\}}{\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}} \quad (\text{loop})$$

Assume $\{\varphi \wedge g\} P \{\varphi\}$ is valid. Need to show that $\{\varphi\} \text{ while } g \text{ do } P \text{ od } \{\varphi \wedge \neg g\}$ is valid.

Recall: $\llbracket \text{while } g \text{ do } P \text{ od} \rrbracket = \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket$

$$\begin{aligned} \llbracket g; P \rrbracket(\langle \varphi \rangle) &= \llbracket P \rrbracket(\langle g \wedge \varphi \rangle) && (\text{see Lemma 2(b)}) \\ &\subseteq \langle \varphi \rangle && (\text{IH}) \end{aligned}$$

$$\text{So } \llbracket g; P \rrbracket^*(\langle \varphi \rangle) \subseteq \langle \varphi \rangle \quad (\text{see Corollary})$$

$$\begin{aligned} \text{So } \llbracket g; P \rrbracket^*; \llbracket \neg g \rrbracket(\langle \varphi \rangle) &= \llbracket \neg g \rrbracket(\llbracket g; P \rrbracket^*(\langle \varphi \rangle)) && (\text{see Lemma 1(c)}) \\ &\subseteq \llbracket \neg g \rrbracket(\langle \varphi \rangle) && (\text{see Lemma 1(a)}) \\ &= \langle \neg g \wedge \varphi \rangle && (\text{see Lemma 2(a)}) \end{aligned}$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} \textcolor{blue}{P} \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} \textcolor{blue}{P} \{\psi'\}} \quad (\text{cons})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\llbracket P \rrbracket(\langle \varphi' \rangle) \subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)})$$

Inductive case 4: Consequence rule

$$\frac{\varphi' \rightarrow \varphi \quad \{\varphi\} P \{\psi\} \quad \psi \rightarrow \psi'}{\{\varphi'\} P \{\psi'\}} \quad (\text{cons})$$

Assume $\{\varphi\} P \{\psi\}$ is valid and $\varphi' \rightarrow \varphi$ and $\psi \rightarrow \psi'$. Need to show that $\{\varphi'\} P \{\psi'\}$ is valid.

Observe: If $\varphi' \rightarrow \varphi$ then $\langle \varphi' \rangle \subseteq \langle \varphi \rangle$

$$\begin{aligned} \llbracket P \rrbracket(\langle \varphi' \rangle) &\subseteq \llbracket P \rrbracket(\langle \varphi \rangle) \quad (\text{see Lemma 1(a)}) \\ &\subseteq \langle \psi \rangle && (\text{IH}) \\ &\subseteq \langle \psi' \rangle \end{aligned}$$

Soundness of Hoare Logic

Theorem

If $\vdash \{\varphi\} P \{\psi\}$ then $\models \{\varphi\} P \{\psi\}$

Summary

- Set theory revisited
- Soundness of Hoare Logic
- Completeness of Hoare Logic

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

⇒ There are true statements that do not have a proof.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.

Incompleteness

Theorem (Gödel's Incompleteness Theorem)

There is no proof system that can prove every valid first-order sentence about arithmetic over the natural numbers.

- ⇒ There are true statements that do not have a proof.
- ⇒ Because of (cons) there are valid triples that result from valid, but unprovable, consequences.
- ⇒ Hoare Logic is not complete.

Relative completeness of Hoare Logic

Theorem (Relative completeness of Hoare Logic)

With an oracle that decides the validity of predicates,

if $\models \{\varphi\} P \{\psi\}$ then $\vdash \{\varphi\} P \{\psi\}$.

Need to know for this course

- Write programs in $\mathcal{L}$.
- Give proofs using the Hoare logic rules (full and outline)
- Definition of $\llbracket \cdot \rrbracket$
- Definition of composition and transitive closure